

✓ Reversing a string:

→ $S1 = \text{"HELLO"}$, reverse of $S1 = \text{"OLLEH"}$.

→ Reverse string needs to swap first character with the last, second character with second last character.

Algorithm:

Step 1: [INITIALIZE] SET $i=0$, $j = \text{Length}(\text{STR}) - 1$

Step 2: Repeat steps 3 & 4 while $i < j$

Step 3: SWAP ($\text{STR}(i)$, $\text{STR}(j)$)

Step 4: SET $i = i + 1$, $j = j - 1$
[END of loop]

Step 5: EXIT

→ Step 1: i is initialized to zero & j is initialized to the length of String - 1

Step 2: While loop is executed until all characters of strings are accessed.

Step 3: Swap i th character of STR with its j th character.

Step 4: Value of i is incremented and j is decremented to traverse STR in forward & backward direction.

• Write a program to reverse the given string:

```
#include <stdio.h>
#include <string.h>
#include <conio.h>

int main()
{
    char str[100], reverse_str[100], temp;
    int i=0, j=0;
    clrscr();
    printf("Enter the string:");
    gets(str);
    j = strlen(str) - 1;
    while (i < j)
    {
        temp = str[j];
        str[j] = str[i];
        str[i] = temp;
```

```
        i++;
        j--;
    }
    printf("In reversed string is:");
    puts(str);
    getch(); return 0;
}
```

output:

Enter the string: Hi there
Reverse string: ereht iH

• Reversing a string:

→ S1 = "HELLO", reverse of S1 = "OLLEH".

→ Reverse string need to swap first character with the last, Second character with second last character.

Algorithm:

Step 1: [INITIALIZE] SET $I=0$, $J=\text{Length}(\text{STR})-1$

Step 2: Repeat steps 3 & 4 while $I < J$

Step 3: SWAP ($\text{STR}(I)$, $\text{STR}(J)$)

Step 4: SET $I=I+1$, $J=J-1$
[END of LOOP]

Step 5: EXIT

→ Step 1: I is initialized to zero & J is initialized to the length of String - 1

Step 2: While loop is executed until all characters of strings are accessed.

Step 3: Swap i^{th} character of STR with its j^{th} character.

Step 4: value of I is incremented and J is decremented to traverse STR in forward & backward direction.

• Write a program to reverse the given string:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
    char str[100], reverse-str[100], temp;
```

```
    int i=0, j=0;
```

```
    clrscr();
```

```
    printf("Enter the string:");
```

```
    gets(str);
```

```
    j = strlen(str) - 1;
```

```
    while (i < j)
```

```
{
    temp = str[j];
```

```
    str[j] = str[i];
```

```
    str[i] = temp;
```

```
        i++;
```

```
        j--;
```

```
}
    printf("In reversed string is:");
```

```
    puts(str);
```

```
    getch(); return 0;
```

```
}
```

output:

Enter the string: Hi there

Reverse string: ereht iH

Extracting a Substring from left:

→ Extract Substring from main string need to copy the content of the string starting from first position to n^{th} position where n is number of character to be extracted.

Algorithm:

Step 1: [Initialize] SET $I=0$

Step 2: Repeat step 3 to 4 while $STR[I] \neq \text{NULL}$ AND $I < N$

Step 3: SET $\text{Substr}[I] = STR[I]$

Step 4: SET $I = I + 1$

[END of loop]

Step 5: SET $\text{Substr}[I] = \text{NULL}$

Step 6: EXIT

write a program to extract the first n character of a string

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int main()
```

```
{ char str[100], substr[100];
```

```
int i=0, n;
```

```
clrscr();
```

```
printf("In enter the string:");
```

```
gets(str);
```

```
printf("In enter the no of character to be copied:");
```

```
scanf("%d", &n);
```

```
i=0;
```

```
while(str[i] != '\0' && i < n)
```

```
{ substr[i] = str[i]; i++;
```

```
}
```

```
substr[i] = '\0';
```

```
printf("In Substring is:");
```

```
puts(substr);
```

```
getch();
```

```
return 0;
```

Output:

Enter the string: Hi there

Enter the no of character to be

copied: 2

Substring is: Hi

Extracting a substring from Right of the string:

- Extracting a substring from right side of main string need to calculate position from the left.
- If $str = \text{"Hello world"}$ and copy 7 characters starting from the right, we have to start extracting character from 4th position.

Algorithm:

- Step 1: [INITIALIZE] SET $i=0$, $j = \text{length}(STR) - N$
- Step 2: Repeat steps 3 to 4 while $STR[j] \neq \text{NULL}$
- Step 3: SET $Substr[i] = STR[j]$
- Step 4: SET $i = i + 1$, $j = j - 1$
[END OF THE LOOP]
- Step 5: SET $Substr[i] = \text{NULL}$
- Step 6: EXIT

write a program to extract the last 'n' characters of a string

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int main()
```

```
{
```

```
char str[100], substr[100];
```

```
int i=0, j=0, n;
```

```
clrscr();
```

```
printf("\n enter the string:");
```

```
gets(str);
```

```
printf("\n enter the no of character to be copied:");
```

```
scanf("%d", &n);
```

```
j = strlen(str) - n;
```

```
while (str[j] != '\0')
```

```
{
```

```
substr[i] = str[j];
```

```
i++, j--;
```

```
}
```

```
substr[i] = '\0';
```

```
printf("\n substring is:");
```

```
puts(substr);
```

```
} getch();
```

```
} return 0;
```

output:

Enter the string : Hi there

Enter the no. of character
to be copied : 5

Substring is: there

Extracting a Substring from the middle of string:

→ Extract Substring from a string requires information about the three things.

→ The main string, position of the first character of substring in the given string. & the number of characters/length of substring.

Ex: `str[] = "welcome to the world of programming";`

then

`Substring (str, 15, 5) = world`

Step 1: [INITIALIZE] Set `I=M`, `J=0`

Step 2: Repeat steps 3 to 6 while `str[I] != NULL` and `N > 0`

Step 3: SET `subStr[J] = str[I]`

Step 4: SET `I = I + 1`

Step 5: SET `J = J + 1`

Step 6: SET `N = N - 1` [END OF LOOP]

Step 7: SET `subStr[J] = NULL`

Step 8: EXIT

write a program to extract a substring from a given string

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int main()
```

```
{
```

```
char str[100], subStr[100];
```

```
int i=0, j=0, n, m;
```

```
clrscr();
```

```
printf("In enter the main string");
```

```
gets(str);
```

```
printf("In enter the position from which to start the substring:");
```

```
scanf("%d", &m);
```

```
printf("In enter the length of the substring:");
```

```
scanf("%d", &n);
```

```
i=m;
```

```
while (str[i] != '\0' && n > 0)
```

```
{
```

```

}
Substr[j] = str[i]; i++; j++; n--;
}
Substr[j] = '\0';
printf("\n Substring is :");
puts(str);
getch();
return 0;
}

```

Output:

(17)

Enter the main string: Hi there

Enter the position from which to

Start the Substring: 1

Enter the length of the
substring: 7

Substring is: i there

Inserting a String in Another string:

→ Insertion operation inserts a string 'S' in the main text T at kth position.

→ The general Syntax of this operation is: INSERT(text, position, String)

Algorithm:

Step 1: [INITIALIZE] SET I=0, J=0 and K=0

Step 2: Repeat Step 3 while text[J] != NULL

Step 3: IF I = pos, then

Repeat while str[k] != NULL

new-str[J] = str[k] SET J = J+1

SET k = k+1

[END of INNER Loop]

Step 4

ELSE

new-str[J] = text[J]

SET J = J+1

[END of If]

SET I = I+1

[END of OUTER Loop]

Step 4: SET new-str[J] = NULL

Step 5: EXIT

write a program to Insert a String in the main text.

• #include <stdio.h>

#include <conio.h>

int main()

{

```

char text[100], str[20], ins_text[100];
int i=0, j=0, k=0, pos;
clrscr();
printf("\n enter the main text:");
gets(text);
printf("\n enter the string to be inserted:");
gets(str);
printf("\n enter the position at which the string has to be inserted:");
scanf("%d", &pos);
while (text[i] != '\0')
{
    if (i == pos)
    {
        while (str[k] != '\0')
        {
            ins_text[j] = str[k];
            j++; k++;
        }
    }
    else
    {
        ins_text[j] = text[i];
        j++;
    }
    i++;
}
ins_text[j] = '\0';
printf("\n new string is:");
puts(ins_text);
getch();
return 0;
}

```

output:

Enter the main text: How you?
 Enter the string to be inserted:
 are

Enter the position at which the string
 has to be inserted: 6

The new string is: How are you?

Deleting a string from main string:

- The deletion operation deletes a substring from a given text
- DELETE (text, position, length)

Ex: Delete ("ABCDXXXABCD", 4, 3) = "ABCDABCD"

Algorithm:

Step 1: [INITIALIZE] SET $I=0$ & $J=0$

Step 2: Repeat steps 3 to 6 while $\text{text}[J] \neq \text{NULL}$

Step 3: IF $I=m$, then

Repeat while $N > 0$

SET $I = I + 1$

SET $N = N - 1$

[END OF LOOP]

[END OF IF]

Step 4: SET $\text{new-str}[J] = \text{text}[J]$

Step 5: SET $J = J + 1$

Step 6: SET $I = I + 1$

[END OF LOOP]

Step 7: SET $\text{new-str}[J] = \text{NULL}$

Step 8: EXIT

Write a program to delete a substring from a text

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int main()
```

```
{
```

```
char text[200], str[20], new-text[200];
```

```
int i=0, j=0, k, n=0, copy-loop=0;
```

```
clrscr();
```

```
printf("\n Enter the main text:");
```

```
gets(text);
```

```
printf("\n enter the string to be deleted");
```

```
gets(str);
```

```
while (text[i] != '\0')
```

```
{ j=0, k=i;
```

```
while (text[k] == str[j] && str[j] != '\0')
```

```
{
```

```
k++; j++;
```

```
}
```

```
if (str[j] == '\0')
```

```
copy-loop = k;
```

```
new-text[n] = text[copy-loop];
```

```
i++; copy-loop++; n++;
```

```
}
```

```

new_str[n] = '\0';
printf("\n the new string is:");
puts(new_text);
getch();
return 0;
}

```

output:

Enter the main text: Hello, how are you?
 Enter the string to be deleted:
 how are you?
 The new string is: Hello

Replacing a pattern with another pattern in a string:

→ Replacement operation is used to replace pattern P1 by another pattern P2

REPLACE (text, pattern1, pattern2)

Algorithm:

Step 1: [INITIALIZE] SET POS = INDEX (Text, P1)

Step 2: SET TEXT = DELETE (TEXT, POS, LENGTH(P1))

Step 3: INSERT (TEXT, POS, P2)

Step 4: EXIT

Write a program to replace a pattern with another pattern in the text:

```

#include <stdio.h>
#include <conio.h>
int main()
{
char str[200], pat[20], new_str[200], rep_pat[100];
int i=0, j=0, k, n=0, copy_loop=0, rep_index=0;
clrscr();
printf("\n enter the string");
gets(str);
printf("\n enter the pattern to be replaced:");
gets(pat);
printf("\n enter the replacing pattern:");
gets(rep_pat);
while (str[i] != '\0')
{
j=0, k=i;

```

```
while (str[k] == pat[j] && pat[j] != '\0')
```

```
{ k++; j++; }
```

```
if (pat[j] == '\0')
```

```
{ copy-loop = k;
```

```
while (rep-pat[rep-index] != '\0')
```

```
{ new-str[n] = rep-pat[rep-index];
```

```
rep-index++;
```

```
n++;
```

```
}
```

```
}
```

```
new-str[n] = '\0';
```

```
printf("In the new string is:");
```

```
puts(new-str);
```

```
getch();
```

```
return 0;
```

```
}
```

output:

Enter the string: How are you?

Enter the pattern to be replaced: ARE

Enter the replacing pattern: are

The new string is:

How are you?

Miscellaneous String and character functions:

character and string manipulation functions that are part of header files - ctype.h, string.h & stdlib.h

Character manipulation function:

Function	Usage.	Example.
isalnum(int c)	Checks whether character c is an alphanumeric character	isalpha('A');
isalpha(int c)	Checks whether character c is an alphabetic character	isalpha('z');
isctrl(int c)	Checks whether character c is a Control character	scanf("%d", &c); isctrl(c);
isdigit(int c)	Checks whether character c is a digit	isdigit(3);
isgraph()	Checks whether character c is a graphic or printing character. The fun excludes the white space characters	isgraph('!');

Function	Usage	Example
isprint(int c)	checks whether character c is a printing character. The function includes the white space character	isprint('@');
islower(int c)	checks whether the character c is in lower case	islower('k');
isupper(int c)	checks whether the character c is in upper case	isupper('K');
ispunct(int c)	checks whether the character 'c' is a punctuation mark	ispunct('?');
isspace(int c)	checks whether the character 'c' is a white space character	isspace(' ');
isxdigit(int c)	checks whether the character 'c' is a hexadecimal digit	isxdigit('F');
tolower(int c)	converts the character c to lower case	tolower('K') return k
toupper(int c)	converts the character c to upper case	tolower('k') return K.

Strcat function:

- The strcat function appends the string pointed to by str2 to the end of the string pointed to by str1
- Terminating null character of str1 is overwritten.
- The process stops when the terminating null character of str2 is copied.
- The argument str1 is returned, str1 should be big enough to store the contents of str2

Syntax:

char *strcat(char *str1, const char *str2);

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
char str1[50] = "programming";
```

```
char str2[] = "In C";
```

```
strcat(str1, str2);
```

```
printf("In str1: %s", str1);
```

```
return 0;
```

```
}
```

Output:

str1: programming in C

strncat function:

- The function appends the string pointed to by str2 to the end of string pointed to by str1 up to n characters long.
- The terminating null character of str1 is overwritten.
- The copying stops until 'n' characters are copied or the terminating null character of str2 is copied.
- Terminating null character is appended to str1 before returning to the calling function

Syntax:

```
char *strncat(char *str1, const char *str2, size_t n);
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{ char str1[50] = "programming";
```

```
char str2[] = "In C";
```

```
strncat(str1, str2, 2);
```

```
printf("In str: %s", str1);
```

```
return 0;
```

```
}
```

Output

str1: programming in

Strchr function:

- The `Strchr()` function searches for the first occurrence of the first occurrence of character `c` in the string pointed to by argument `str`.
- The function returns a pointer pointing to the first matching character, or `NULL` if no match found.

Syntax:

```
char *strchr (const char *str, int c);
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main ()
```

```
{
```

```
char str[50] = "programming in c";
```

```
char *pos;
```

```
pos = strchr(str, 'n');
```

```
if (pos)
```

```
printf("In n is found in str at position %d", pos);
```

```
else
```

```
printf("In n is not present in the string");
```

```
return 0;
```

```
}
```

output

n is found in str
at position 10

• Strrchr function:

- The `Strrchr()` function searches first ~~comes~~ occurrence of the character `c` beginning at the rear end and working towards the front in the string pointed to by the argument `str`.
- The function searches for last occurrence of character `c` and returns a pointer pointing to the last matching character or `NULL` if no match is found.

Syntax: `char *strrchr(const char *str, int c);`

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str[50] = "Programming in C";
    char *pos;
    pos = strrchr(str, 'n');
    if (pos)
        printf("In the last position of n is: %d", pos);
    else
        printf("In n is not present in the string");
    return 0;
}
```

output:

The last position of n
is: 13

strcmp function:

- The strcmp function compares the string pointed to by str1 to the string pointed to by str2.
- The function returns zero if the strings are equal.
- It returns a value less than zero or greater than zero if str1 is less than or greater than str2

Syntax: `int strcmp(const char *str1, const char *str2);`

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str1[10] = "HELLO";
    char str2[10] = "HEY";
```

output:

Two strings are not
identical.

```
if (strcmp(str1, str2) == 0)
    printf("In two strings are identical");
else
    printf("In two strings are not identical");
return 0;
}
```

Strncmp function:

- The function compares ~~not~~ most first n bytes of $str1$ and $str2$
- The process stops comparing after null character is encountered
- The function return zero if the first n bytes of the string are equal.
- It returns a value less than zero or greater than zero if $str1$ is less than or greater than $str2$

Syntax: `int strncmp(const char *str1, const char *str2, size_t n);`

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
    char str1[10] = "HELLO";
```

```
    char str2[10] = "HEY";
```

```
    if (strncmp(str1, str2, 2) == 0)
```

```
        printf("In two strings are identical");
```

```
    else
```

```
        printf("In two strings are not identical");
```

```
    return 0;
```

```
}
```

Output:

The two strings are identical

strcpy function:

- The function copies the string pointed to by $str2$ to $str1$ including null character of $str2$.
- It returns the argument $str1$, here $str1$ should be big enough to store the contents of $str2$

Syntax: `char *strcpy(char *str1, const char *str2);`

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{ char str1[10], str2[10] = "HELLO";
```

output:

HELLO

```
strcpy(str1, str2);
```

```
printf("\n str1: %s", str1);
```

```
return 0;
```

```
}
```

strcpy function:

- The function copies up to n characters from string pointed to by $str2$ to $str1$
- The copying stop when n characters are copied.
- The null character is continually copied to $str1$ until n characters have been copied, null character is appended to $str1$
- If n is zero or negative nothing is copied

Syntax: strcpy function

```
char *strcpy ( char *str1, const char *str2, size_t n);
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{ char str1[10], str2[10] = "HELLO";
```

```
strcpy(str1, str2, 2);
```

```
printf("\n str1: %s", str1);
```

```
return 0;
```

```
}
```

output:

HE

strlen function:

- The function calculates length of the string str up to but not including null character, i.e. function returns number of characters in the string.

Syntax:

```
size_t strlen ( const char *str);
```

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str[] = "HELLO";
    printf("Length of str is : %d", strlen(str));
    return 0;
}
```

output:

length of str is : 5

strstr function:

- The function is used to find first occurrence of string str2 in the string str1.
- It returns a pointer to the first occurrence of str2 in str1.
- If no match is found, null pointer is returned.

Syntax:

```
char *strstr(const char *str1, const char *str2);
```

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str1[] = "HAPPY BIRTHDAY TO YOU";
    char str2[] = "DAY";
    char *ptr;
    ptr = strstr(str1, str2);
    if (ptr)
        printf("Substring found");
    else
        printf("Substring not found");
    return 0;
}
```

output

Substring found

Strspn function:

The function return the index of the first character in str1 that doesn't match any character in str2

Syntax:

```
Size_t Strspn (const char *str1, const char *str2);
#include <stdio.h>
#include <string.h>
int main()
{
    char str1[] = "HAPPY BIRTHDAY TO YOU";
    char str2[] = "HAPPY BIRTHDAY JOE";
    printf("\n the position of first character in str2 that does not
        match with that in str1 is '%d'", Strspn(str1, str2));
    return 0;
}
```

output:

position of first character in str2 that does not match with that in str1 is 15.

Strcspn function:

The function returns the index of first character in str1 that matches any of the characters in str2

Syntax:

```
Size_t Strcspn (const char *str1, const char *str2);
#include <stdio.h>
#include <string.h>
int main()
{
    char str1[] = "Programming in C";
    char str2[] = "C";
    printf("\n position of first character in str2 that matches with
        that in str1 is '%d'", Strcspn(str1, str2));
    return 0;
}
```

output

position of first character in str2 that matches with str1 is 8

Strpbrk function:

(39)

- The function `strpbrk()` returns a pointer to the first occurrence in `str1` of any character in `str2`, NULL if none are present.
- The difference between `strpbrk()` and `strcspn` is that `strcspn()` returns index of character & `strpbrk()` returns a pointer to first occurrence of character in `str2`

Syntax:

`char *strpbrk (const char *str1, const char *str2);`

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{ char str1[] = "programming in c";
```

```
  char str2[] = "AB";
```

```
  char *ptr = strpbrk (str1, str2);
```

```
  printf ("In the position of first character in str2 that does not  
    match with that in str1 is %d", strcspn(str1, str2));
```

```
  return 0;
```

```
}
```

output:

no character matches in the two string.

Arrays of Strings:

→ The String is an array of characters

Ex: `char name[] = "mohan"`, name is a string that has five characters

There are 20 students in a class, array of strings would store 20 individual strings

`char names[20][30];`

→ First index that specifies the number of strings that are needed and second index specifies the length of every individual string.

→ It allocate space for 20 names where each name can be a maximum of 30 character long.

→ The general syntax for declaring a two dimensional array of strings can be given

`<data type> <array-name> [row-size][column-size];`

→ The memory representation of an array of strings.

`char name[5][10] = {"Ram", "mohan", "shyam", "Hari", "Gopal"};`

→ The array names allocate 50 bytes. but actual memory occupied ~~allocated~~ is 27 bytes. more than half of the memory allocated lies wasted.

Step 1 : [INITIALIZE] SET `I=0`

Step 2 : Repeat Step 3 while `I < N`

Step 3 : Apply process to `NAMES[I]`
[END OF LOOP]

Step 4 : EXIT

Programs :

Write a program to read & print the names of `n` students of a class.

```

#include <stdio.h>
#include <conio.h>
int main()
{
    char names[5][10];
    int i, n;
    clrscr();
    printf("\n Enter the number of Students:");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        printf("\n Enter the name of student %d:", i+1);
        gets(names[i]);
    }
    getch();
    printf("\n names of student are :\n");
    for(i=0; i<n; i++)
        puts(names[i]);
    getch(); return 0;
}

```

output:

Enter the no of Students: 3
 Enter the name of student 1:
 Aditya
 Enter the name of student 2:
 Gauravsh
 Enter the name of student 3:
 Sarthak
 Names of Students are:
 Aditya Gauravsh
 Sarthak

Write a program to Sort names of Students:

```

#include <stdio.h>
#include <conio.h>
int main()
{
    char names[5][10], temp[10];
    int i, n, j;
    clrscr();
    printf("\n Enter the number of Students:");
    scanf("%d", &n);
    for(i=0; i<n; i++)
    {
        printf("\n Enter the name of the student %d:", i+1);
        gets(names[i]);
    }
    for(i=0; i<n; i++)
    {
        for(j=0; j<n-i-1; j++)
        {

```

```

if (strcmp(names[j], names[j+1]) > 0)
{
    strcpy(temp, names[j]);
    strcpy(names[j], names[j+1]);
    strcpy(names[j+1], temp);
}
}
printf("In names of the student are:");
for (i=0; i<n; i++)
    puts(name[i]);
return 0;
}

```

output

Enter the no of students: 3
 Enter name of student 1: Sarthak
 Enter name of student 2: Gauransh
 Enter name of student 3: Aditya
 Name of students are:
 Aditya, Gauransh, Sarthak

write a program to read a Sentence, then count the number of words in the Sentence:

```

#include <stdio.h>
#include <conio.h>
int main()
{
    char str[200];
    int i=0; count=0;
    clrscr();
    printf("In Enter the Sentence:");
    gets(str);
    while (str[i] != '\0')
    {
        if (str[i] == ' ' && str[i+1] != ' ')
            count++;
        i++;
    }
    printf("\n total count of word is: %d", count+1);
    return 0;
}

```

o/p

Enter the Sentence: How are you
 total count of words is: 3

(34)

Write a program to find whether a given string is a palindrome or not

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int main()
```

```
{
```

```
    char str[100];
```

```
    int i=0; j, length=0;
```

```
    clrscr();
```

```
    printf("\n Enter the string: ");
```

```
    gets(str);
```

```
    while (str[i] != '\0')
```

```
    {
```

```
        length++;
```

```
        i++;
```

```
    }
```

```
    i=0;
```

```
    j=length-1;
```

```
    while (i <= length/2)
```

```
    { if (str[i] == str[j])
```

```
    {
```

```
        i++;
```

```
        j--;
```

```
    }
```

```
    else
```

```
        break;
```

```
    }
```

```
    if (i >= j)
```

```
        printf("\n PALINDROME");
```

```
    else
```

```
        printf("\n NOT A PALINDROME");
```

```
    return 0;
```

```
}
```

Output

Enter the string: madam

Palindrome